Aplicações de Processamento Digital de Sinais – 4456S-04

Prática 7: Compressão de Imagem

Antônio Vinicius Corrêa dos Santos

13/10/2026

Imagem: `lena_c.jpg`, 256×256, convertida para escala de cinza com `rgb2gray` e mantida em $[0, 255]$. Todas as figuras são geradas por `main.m` e gravadas em `plots/`.

1 Espectro da DCT 2D

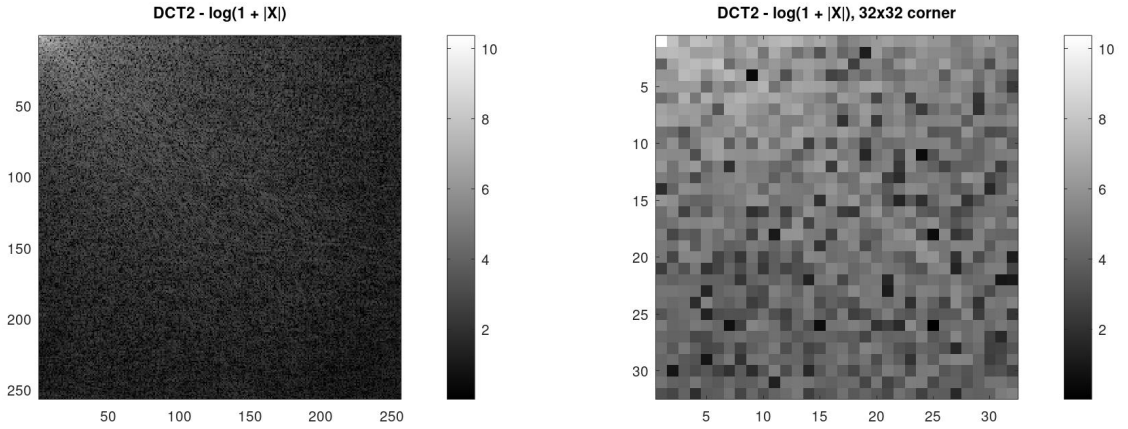


Figura 1: Espectro de magnitude da DCT, $\log(1 + |X(u, v)|)$: matriz completa e canto 32×32 de baixa frequência.

Os pontos mais brilhantes estão no **canto superior esquerdo** da matriz, em torno de $(u, v) = (1, 1)$, e a amplitude cai ao longo das diagonais em direção ao canto inferior direito. O coeficiente $(1, 1)$ é a componente DC, proporcional ao brilho médio da imagem; os vizinhos são as baixas frequências espaciais, que descrevem as variações lentas de intensidade — regiões homogêneas, sombreamento e a forma geral dos objetos. Os coeficientes distantes do canto representam as variações rápidas (bordas finas, textura do cabelo e do chapéu, ruído) e têm amplitude muito menor.

Diferente da DFT, a DCT não precisa de `fftshift`: ela não tem frequências negativas, então a origem já é o canto, e como a extensão implícita da imagem é par, não aparecem as faixas causadas pela descontinuidade entre bordas opostas vistas na Prática 6.

2 Máscara zonal triangular

A máscara A vale 1 no triângulo superior esquerdo de lado j, ou seja, nas posições com $u + v \leq j + 1$, e 0 nas demais:

```
function A = zonal_mask(j, N)
    A = zeros(N);
    A(1:j,1:j) = fliplr(triu(ones(j)));
end
```

`triu` gera o triângulo acima da diagonal principal e `fliplr` o espelha para que a hipotenusa fique na antidiagonal. O número de coeficientes mantidos é $j(j+1)/2$.

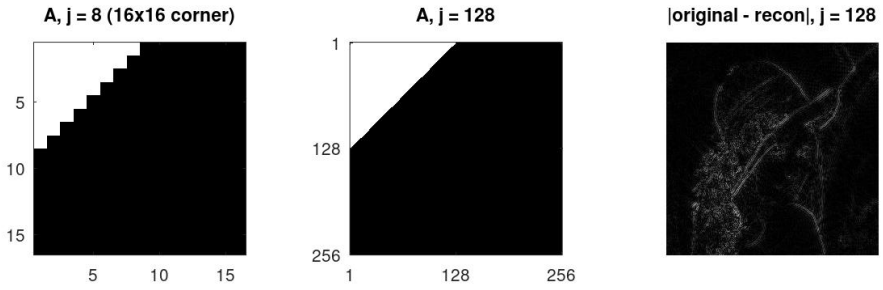


Figura 2: Máscara para $j = 8$ (canto 16×16), máscara para $j = 128$ e erro absoluto da reconstrução com $j = 128$.

3 Reconstrução com coeficientes truncados

Para cada $j \in \{1, 2, 4, 8, 16, 32, 64, 128\}$: $Y = \text{idct2}(X \odot A_j)$. As reconstruções em tamanho original estão em `plots/recon_j-*.jpg`.



Figura 3: Original e reconstruções para os oito valores de j.



Figura 4: Reconstruções para $j = 16$, 32 e 128, a partir dos arquivos em resolução original (256×256).

Tabela 1: Coeficientes mantidos, fração da energia retida $\sum |X \odot A|^2 / \sum |X|^2$ e erro em relação à original.

j	**Coef.**	**Mantidos (%)**	**Energia**	**RMSE**	**PSNR (dB)**
1	1	0,0015	0,8708	47,77	14,55
2	3	0,0046	0,8794	46,15	14,85
4	10	0,0153	0,8913	43,83	15,30
8	36	0,0549	0,9202	37,55	16,64
16	136	0,2075	0,9527	28,90	18,91
32	528	0,8057	0,9740	21,44	21,51
64	2080	3,1738	0,9871	15,07	24,57
128	8256	12,5977	0,9946	9,74	28,36

Observações:

- $j = 1$ **a** 4**.** Com só o termo DC a imagem é um cinza uniforme no nível médio; até $j = 4$ aparecem apenas gradientes suaves. Ainda assim o termo DC sozinho carrega 87 % da energia — energia não é o mesmo que informação visual.
- $j = 8$ **e** 16**.** Surgem manchas claras e escuras na posição do rosto e do chapéu, mas sem forma definida.
- **A partir de** $j = 32$ as características principais tornam-se discerníveis: o contorno do chapéu, o rosto, o ombro e a moldura ao fundo são reconhecíveis, embora borrados. Nesse ponto são mantidos $32 \cdot 33/2 = 528$ coeficientes, isto é, $528/65536 \approx 0{,}81\,\%$ do total.
- $j = 64$ já mostra os olhos, a boca e a pena do chapéu com 3,2 % dos coeficientes.
- O borrado das reconstruções tem um leve *ringing* junto às bordas fortes, efeito do corte abrupto da máscara (o mesmo fenômeno do passa-baixas ideal da Prática 6).

4 Comparação entre $j = 128$ e a original

Com $j = 128$ são descartados 87,4 % dos coeficientes e, à distância normal de visualização, a imagem é praticamente indistinguível da original: PSNR de 28,4 dB e 99,46 % da energia retida. A perda existe, mas é sutil: uma leve suavização da textura do cabelo e da pena, e o mapa de erro mostra que ela está concentrada exatamente nessas bordas e texturas finas, enquanto as regiões homogêneas (pele, fundo) têm erro quase nulo.

A justificativa é a **compactação de energia** da DCT. Pixels vizinhos de uma imagem natural são fortemente correlacionados, e a DCT se aproxima da transformada ótima (Karhunen–Loève) para esse tipo de sinal: ela concentra quase toda a energia em poucos coeficientes de baixa frequência. Os 87 % descartados respondem por apenas 0,54 % da energia, e correspondem a detalhes de alta frequência aos quais o olho humano é pouco sensível. É esse o princípio explorado pelo JPEG, que aplica a DCT em blocos 8×8 e quantiza com mais força os coeficientes de alta frequência.

5 DCT em blocos com `blockproc`

A imagem foi dividida em blocos $B \times B$ e a DCT aplicada em cada bloco independentemente, com `funcao_dct = @(b) dct2(b.data)` e, na volta, `funcao_idct = @(b) idct2(b.data)`. No Octave o `blockproc` entrega o bloco como matriz, e não como *struct* com campo `data`; a função auxiliar `bdata` aceita os dois formatos, de modo que o script roda igual no MATLAB.

Pixels da imagem original e transformada ($B = 8$)

O primeiro bloco 8×8 (canto superior esquerdo, região do fundo) e sua DCT:

$$x = \begin{bmatrix} 161 & 161 & 162 & 163 & 164 & 163 & 160 & 158 \\ 162 & 161 & 162 & 162 & 162 & 161 & 158 & 156 \\ 161 & 161 & 161 & 161 & 161 & 159 & 157 & 154 \\ 159 & 159 & 159 & 159 & 159 & 158 & 156 & 154 \\ 157 & 156 & 156 & 157 & 158 & 158 & 156 & 155 \\ 156 & 155 & 155 & 156 & 157 & 158 & 157 & 156 \\ 156 & 156 & 155 & 156 & 157 & 157 & 156 & 155 \\ 158 & 157 & 156 & 156 & 156 & 156 & 155 & 154 \end{bmatrix} \qquad X = \begin{bmatrix} 1264 & 6 & -6 & 5 & 0 & 0 & 0 & 0 \\ 16 & 4 & -4 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -6 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(valores arredondados). Os pixels variam pouco em torno de 158, e a DCT reflete isso: o termo DC vale $1264 = 8 \cdot \bar{x}$ ($\bar{x} \approx 158$) e quase todos os demais coeficientes são próximos de zero. A compactação de energia observada na imagem inteira se repete dentro de cada bloco. Na imagem transformada completa (Figura 5, à esquerda) cada bloco tem seu ponto brilhante no canto superior esquerdo, formando uma grade com a mesma periodicidade dos blocos.

Zerando coeficientes

Em cada bloco foram mantidos apenas os coeficientes do triângulo $u + v \leq B/2 + 1$ (a mesma máscara zonal da seção anterior, com $j = B/2$) e o resto foi zerado. No primeiro bloco isso remove só o -6 e o 1 da quarta linha e os -1 da sexta; a reconstrução fica

$$\hat{x} = \begin{bmatrix} 162 & 162 & 163 & 163 & 164 & 162 & 159 & 157 \\ 161 & 161 & 162 & 162 & 163 & 161 & 159 & 156 \\ 160 & 160 & 160 & 161 & 161 & 160 & 158 & 155 \\ 159 & 158 & 158 & 159 & 159 & 159 & 157 & 155 \\ 157 & 157 & 157 & 157 & 158 & 157 & 156 & 154 \\ 157 & 156 & 156 & 156 & 157 & 157 & 156 & 154 \\ 157 & 156 & 155 & 156 & 157 & 157 & 156 & 155 \\ 157 & 156 & 155 & 156 & 157 & 157 & 156 & 155 \end{bmatrix},$$

com erro de no máximo ± 2 níveis de cinza por pixel. Zerar um coeficiente da DCT não apaga um pixel: cada coeficiente é uma função-base cosseno que cobre o bloco inteiro, então o erro se espalha de forma suave por todos os 64 pixels do bloco, e nenhum pixel fora dele é afetado.

Sem zerar nada, a IDCT recupera a imagem original até o erro numérico ($\max |\hat{x} - x| < 10^{-12}$), confirmando que a DCT em blocos é inversível.

Variação do tamanho do bloco

Os pixels foram verificados novamente na mesma janela 8×8 do canto superior esquerdo, agora para $B = 4$, 16 e 32. Na imagem transformada:

$$X_{B=4} = \begin{bmatrix} 644 & -1 & 1 & 0 & 635 & 9 & -1 & 0 \\ 4 & -1 & 0 & 0 & 7 & 1 & 0 & 0 \\ -2 & -1 & 0 & 0 & 1 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 625 & 1 & 2 & 0 & 625 & 3 & -1 & -1 \\ 0 & -1 & 0 & 0 & 2 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$X_{B=32} = \begin{bmatrix} 4971 & 138 & -144 & 239 & -178 & 62 & -105 & 89 \\ 40 & -30 & -60 & -31 & 15 & 53 & 27 & -29 \\ -36 & -40 & 93 & 90 & -43 & -45 & -7 & 2 \\ 40 & 32 & -17 & -55 & -18 & 48 & 25 & -4 \\ 0 & -10 & 12 & 19 & 25 & 1 & -33 & -7 \\ 10 & 2 & 5 & -20 & -8 & -4 & 6 & 6 \\ -1 & -1 & 7 & 8 & 12 & -6 & -4 & 0 \\ 14 & 8 & -5 & -13 & -2 & -3 & 6 & 0 \end{bmatrix}$$

- Com $B = 4$ a janela contém quatro blocos inteiros, e aparecem quatro termos DC (644, 635, 625, 625), cada um igual a 4 vezes a média do seu bloco ($\approx$ 161, 159, 156, 156). Os outros coeficientes são quase nulos: num bloco tão pequeno de fundo quase não há variação.
- Com $B = 16$ e 32 a janela mostra só o canto de baixa frequência de um único bloco. O DC cresce com o bloco ($2516 = 16 \cdot 157{,}3$ e $4971 = 32 \cdot 155{,}3$), e os coeficientes AC deixam de ser desprezíveis (até ≈ 240 para $B = 32$), porque o bloco já abrange o fundo, a moldura e parte do chapéu.
- Com $B = 32$ a janela inteira está dentro do triângulo mantido ($u + v \leq 17$), então nenhum dos coeficientes mostrados é zerado; o que se perde fica fora dela.

Na imagem reconstruída após zerar os coeficientes, os pixels da janela continuam próximos da original para todos os tamanhos. O erro máximo nessa janela foi 1,42, 1,55, 1,88 e 2,34 níveis de cinza para $B = 4$, 8, 16 e 32. Ele cresce um pouco com B porque o erro de cada bloco se espalha por mais pixels, mas continua imperceptível nessa região de fundo.

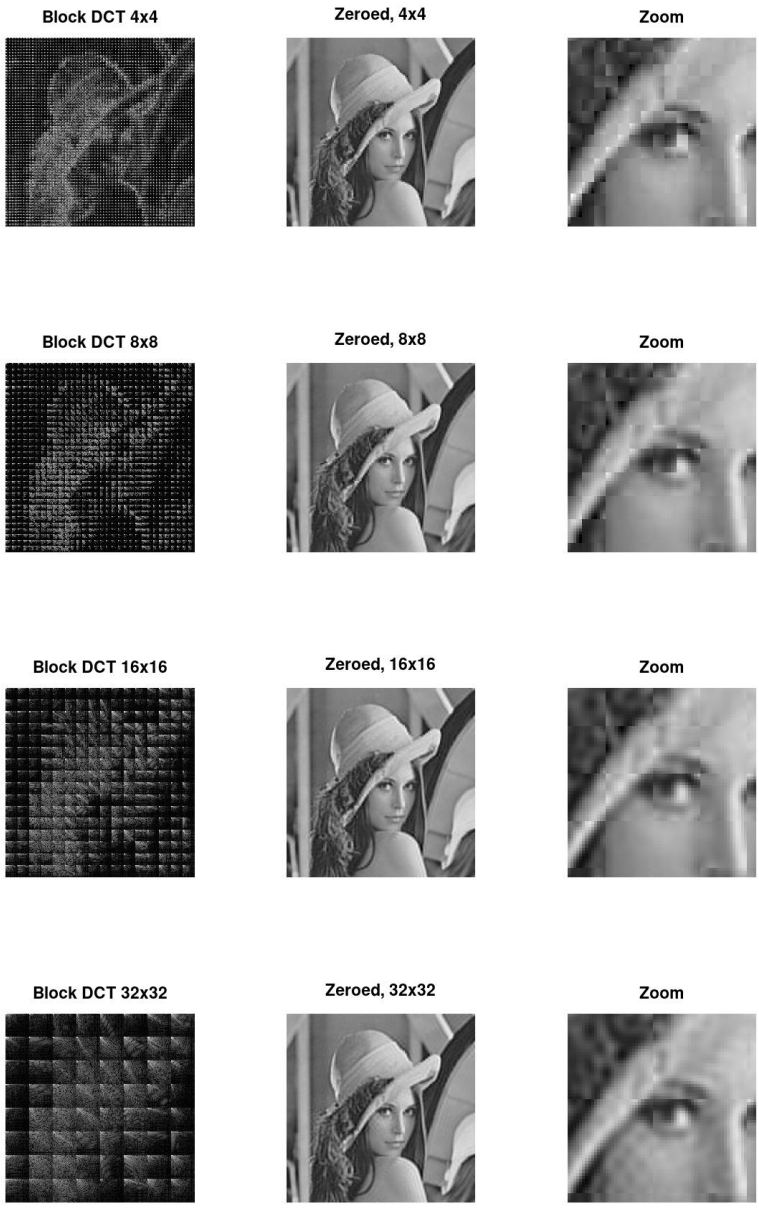


Figura 5: Para $B = 4, 8, 16, 32$: DCT em blocos ($\log(1 + |X|)$), reconstrução após zerar os coeficientes fora do triângulo $j = B/2$ e *zoom* na região do olho.

Tabela 2: Reconstrução com coeficientes zerados, por tamanho de bloco.

B	**Mantidos (%)**	**RMSE**	**PSNR (dB)**
4	18,75	9,94	28,18
8	15,62	9,89	28,23
16	14,06	9,84	28,27
32	13,28	9,81	28,29

- Na imagem transformada a grade acompanha B: com blocos pequenos ela parece uma versão em mosaico da própria imagem (cada termo DC é a média do bloco); com blocos grandes cada bloco mostra um espectro de verdade, com a energia caindo do canto para

fora.

- Blocos maiores mantêm uma fração menor dos coeficientes e ainda assim dão erro um pouco menor: há mais pixels correlacionados por bloco, então a compactação de energia é melhor.
- O tipo de artefato muda com B. Com blocos pequenos (4, 8) aparece o *efeito de bloco*: cada bloco é aproximado sozinho, e as bordas entre blocos vizinhos ficam visíveis como degraus no *zoom*. Com blocos grandes (16, 32) as fronteiras somem, mas volta o *ringing* junto às bordas fortes, como na DCT da imagem inteira.
- O custo computacional também cresce com B, e blocos grandes misturam regiões de conteúdo diferente. O JPEG adota 8×8 como compromisso entre compactação, custo e localização dos artefatos.

Script (Octave)

```
close all
clear all
clc

pkg load signal
pkg load image

global VIS = 'off';

if ~exist('plots', 'dir'), mkdir('plots'); end

fig = 'lena_c.jpg';

% ------------------------------------------------------------------ helpers

% Spec asks for gray levels in [0,255], so no normalization here.
function img = read_img(name)
    root = './img/';
    X = imread([root name]);
    X = rgb2gray(X);
    img = double(X);
end

function save_fig(tag)
    saveas(gcf, ['plots/' tag '.jpg']);
end

% Zonal mask: ones on the upper-left j x j triangle (u + v <= j + 1).
function A = zonal_mask(j, N)
    A = zeros(N);
    A(1:j,1:j) = fliplr(triu(ones(j)));
end

% ================================================================== items 1-4

img = read_img(fig);
N   = rows(img);
X   = dct2(img);

figure(1, 'visible', VIS);
colormap(gray(256));
imagesc(log(1 + abs(X)));
axis image;
colorbar;
title('DCT2 - log(1 + |X|)');
save_fig('dct_spectrum');

% Zoom on the low-frequency corner, where the energy lives.
figure(2, 'visible', VIS);
colormap(gray(256));
imagesc(log(1 + abs(X(1:32,1:32))));
axis image;
colorbar;
```

```
54 title('DCT2 - log(1 + |X|), 32x32 corner');
55 save_fig('dct_spectrum_zoom');
56
57 % ================================================================ items 5-9
58
59 js = [1 2 4 8 16 32 64 128];
60
61 figure(3, 'visible', VIS);
62 colormap(gray(256));
63 subplot(3,3,1); imshow(img, [0 255]); title('Original');
64
65 printf('%5s %8s %10s %10s %8s %8s\n', 'j', 'coefs', 'kept (%)', 'energy', 'RMSE', 'PSNR')
       ;
66 for k = 1:numel(js)
67     j = js(k);
68     A = zonal_mask(j, N);
69     Y = idct2(X .* A);
70
71     subplot(3,3,k+1); imshow(Y, [0 255]); title(sprintf('j = %d', j));
72     % Full-resolution copy of each reconstruction (the truesize view).
73     imwrite(uint8(Y), sprintf('plots/recon_j-%03d.jpg', j));
74
75     n_kept = sum(A(:));
76     energy = sum(sum((X .* A).^2)) / sum(X(:).^2);
77     rmse   = sqrt(mean((Y(:) - img(:)).^2));
78     printf('%5d %8d %10.4f %10.6f %8.2f %8.2f\n', ...
79            j, n_kept, 100 * n_kept / N^2, energy, rmse, 20*log10(255/rmse));
80 end
81 set(gcf, 'paperposition', [0 0 6 6.6]);
82 save_fig('recon_grid');
83
84 figure(4, 'visible', VIS);
85 colormap(gray(256));
86 subplot(1,3,1); imshow(zonal_mask(8, N)(1:16,1:16));   axis on; title('A, j = 8 (16x16
       corner)');
87 subplot(1,3,2); imshow(zonal_mask(128, N));            axis on; title('A, j = 128');
88 set(gca, 'xtick', [1 128 256], 'ytick', [1 128 256]);
89 subplot(1,3,3); imshow(abs(img - idct2(X .* zonal_mask(128, N))), []);
90 title('|original - recon|, j = 128');
91 set(gcf, 'paperposition', [0 0 9 3]);
92 save_fig('masks_and_error');
93
94 % ================================================================ block DCT
95
96 % MATLAB's blockproc hands the function a struct (b.data); Octave's hands the
97 % raw block. This keeps the spec's @(b) idct2(b.data) form working in both.
98 function d = bdata(b)
99     if isstruct(b), d = b.data; else, d = b; end
100 end
101
102 funcao_dct  = @(b) dct2(bdata(b));
103 funcao_idct = @(b) idct2(bdata(b));
104
105 printf('\nOriginal, first 8x8 pixels:\n');
106 disp(img(1:8,1:8));
107
108 for B = [4 8 16 32]
109     T = blockproc(img, [B B], funcao_dct);
110
111     % Same zonal triangle as before, inside each block: keep u + v <= B/2 + 1.
112     M  = zonal_mask(B/2, B);
113     Tz = blockproc(T, [B B], @(b) bdata(b) .* M);
114
115     R  = blockproc(T,  [B B], funcao_idct);
116     Rz = blockproc(Tz, [B B], funcao_idct);
117
118     % Same 8x8 window for every B: holds 4 blocks when B = 4, part of one otherwise.
119     printf('\n--- B = %d ---\n', B);
120     printf('Transformed, first 8x8:\n');                    disp(round(T(1:8,1:8)));
121     printf('Zeroed, first 8x8:\n');                         disp(round(Tz(1:8,1:8)));
122     printf('Reconstructed from zeroed, first 8x8 pixels:\n'); disp(round(Rz(1:8,1:8)));
123     printf('max |error| in that window: %.2f\n\n', max(max(abs(Rz(1:8,1:8) - img(1:8,1:8)
        ))));
```

```
124
125     rmse = sqrt(mean((Rz(:) - img(:)).^2));
126     printf('%5s %10s %12s %8s %8s\n', 'B', 'kept (%)', 'max|rec-x|', 'RMSE', 'PSNR');
127     printf('%5d %10.2f %12.2e %8.2f %8.2f\n', B, 100 * sum(M(:)) / B^2, ...
128            max(abs(R(:) - img(:))), rmse, 20*log10(255/rmse));
129
130     figure(10+B, 'visible', VIS);
131     colormap(gray(256));
132     subplot(1,3,1); imagesc(log(1 + abs(T))); axis image off;
133     title(sprintf('Block DCT %dx%d', B, B));
134     subplot(1,3,2); imshow(Rz, [0 255]); title(sprintf('Zeroed, %dx%d', B, B));
135     % Zoom on the eye region, where block edges are easiest to see.
136     subplot(1,3,3); imshow(Rz(100:163,100:163), [0 255]); title('Zoom');
137     % 1x3 layout: a short page avoids the empty bands above and below.
138     set(gcf, 'paperposition', [0 0 9 3]);
139     save_fig(sprintf('block_%02d', B));
140 end
```

Listing 1: `main.m`